Python Exception Handling

A Beginner's Guide with Examples

Object-Oriented Programming • Python 3

What is Exception Handling?

When a Python program runs, things can go wrong — a user types a letter instead of a number, or the code tries to divide by zero. Without exception handling, the program simply crashes and stops.

Exception handling lets your program catch these problems, deal with them gracefully, and continue running — instead of crashing.

Errors in Python come in two types:

- Syntax Errors — mistakes in the code's grammar (wrong indentation, missing colon, etc.). Python catches these before the program even runs.
- Exceptions — errors that happen while the program is running, even though the code is written correctly.

💡 Think of exception handling like a safety net under a tightrope walker. If they slip (an error occurs), the net catches them (the except block) so they don't hit the ground (crash the program).

Part 1: Syntax Error vs Exception

Syntax Error — NameError Example

This code is syntactically correct (Python can read it), but b was never defined. Python raises a NameError at runtime.

```
a = 10
if a == 10:
    print(b)   # b is not defined anywhere!
```

```
NameError: name 'b' is not defined
```

Exception — ZeroDivisionError Example

Dividing by zero is mathematically impossible. Python raises a ZeroDivisionError when this happens at runtime.

```
100 / 0
ZeroDivisionError: division by zero
```

Here's a fuller example showing that code before the error still runs — but anything after it does not:

```
marks = 10000
print("Hi")        # This prints successfully
a = marks / 0      # Error happens here
print(a)           # This never runs
print("Hello")     # This never runs either
Hi
ZeroDivisionError: division by zero
```

⚠️ Notice that 'Hi' printed, but 'Hello' never did. The program stopped at the line with the error. Exception handling solves exactly this problem.

Part 2: try and except

The try block contains code that might cause an error. If an error occurs, Python jumps to the matching except block instead of crashing.

```
try:
    # Code that might fail
    risky_code()
except SomeError:
    # What to do if that error occurs
    print("Something went wrong")
```

Example — divide() Function

This function safely handles three different types of errors that could occur during division:

```
def divide(a, b):
    try:
        return a / b
```

```
    except ZeroDivisionError:
        return "Division by zero is not allowed"
    except TypeError:
        return "Use Integers inside the function"
    except ValueError:
        return "Use only integer data"
```

Test 1 — Division by zero

```
print("Hi")
print(divide(1, 0))
print("Hello")
print("ok")
```

```
Hi
Division by zero is not allowed
Hello
ok
```

✅ All four lines printed! The error was caught by the except block and returned a message — the program kept running normally.

Test 2 — Wrong data type

```
print(divide("s", "g"))
print("Hello")
print("ok")
```

```
Use Integers inside the function
Hello
ok
```

Part 3: try, except, else, and finally

Python provides four keywords for complete exception handling:

Keyword	When It Runs
`try`	Always — contains the code that might fail.
`except`	Only if an error occurs in the try block.
`else`	Only if NO error occurred in the try block.
`finally`	Always — whether there was an error or not. Used for cleanup.

Example 1 — Full exception_handling() with finally

```
def exception_handling():
    try:
        a = int(input("Enter a number: "))
        b = int(input("Enter another number: "))
        print(a / b)
    except ZeroDivisionError:
        print("You can't divide by zero")
    except ValueError:
        print("You must enter a number")
    except:
        print("Something went wrong")
    else:
        print("Nothing went wrong")   # Runs only if no error
    finally:
        if a == 10:
            print("This is our final block")
        else:
            print("Siva")

exception_handling()
```

Output (when user enters 10 and 30)

```
Enter a number:  10
Enter another number:  30
0.3333333333333333
Nothing went wrong
This is our final block
```

💡 Both else and finally ran here because no error occurred. The else block confirmed success, and the finally block always runs no matter what.

Example 2 — Squaring a Number

```
def exception_handling(n):
    try:
        print(n ** 2)
    except NameError:
        print("Undefined variable")
```

```
    except TypeError:
        print("Use numbers only")
    except:
        print("Something went wrong")
    else:
        print("There is no Error")

exception_handling(77)    # Valid number
exception_handling("s")   # Invalid - string
```

Output

```
5929
There is no Error
Use numbers only
```

💡 When 77 was passed, the try block succeeded, so else ran too. When "s" was passed, TypeError was raised and caught — else did NOT run.

Part 4: raise — Triggering Exceptions Manually

Sometimes you want to deliberately trigger an exception when certain conditions are met — for example, if a user enters an invalid value that is technically valid Python but logically wrong (like a negative age).

The raise keyword lets you manually throw an exception with a custom message.

Example 1 — Negative Age (with try/except)

```
age = int(input("Enter your age: "))

try:
    if age < 0:
        raise ValueError("Age cannot be negative")
    else:
        print("Your age is", age)
except ValueError as e:
    print("error", e)
```

Output (when user enters -9)

```
Enter your age: -9
error Age cannot be negative
```

✅ The ValueError was manually raised using raise. The except block caught it and printed the custom message. The program did not crash!

Example 2 — Negative Age (without try/except)

If you use raise without a try/except to catch it, the program will crash with the error:

```
age = -4
if age < 0:
    raise ValueError("Age cannot be negative")
else:
    print("Your age is", age)
ValueError: Age cannot be negative
```

⚠️ Always wrap raise inside a try/except if you want the program to continue running after the error is triggered.

Example 3 — List Index Validation

This example raises an IndexError if the user asks for an index that is out of range:

```
list1 = [3, 6, 9, 34, 66, 123, 456]
num = int(input("Enter index number to get the value: "))

try:
    if num > len(list1) - 1:
        raise IndexError("The index should be inside the range of values")
    else:
        output = list1[num]
        print("Your index value is:", output)
except IndexError as e:
    print(f"Error: {e}")
```

Output (when user enters a valid index, e.g. 2)

```
Your index value is: 9
```

Output (when user enters an out-of-range index, e.g. 10)

```
Error: The index should be inside the range of values
```

Common Python Exceptions

Exception	When It Occurs
`ZeroDivisionError`	When you try to divide a number by zero.
`NameError`	When you use a variable that has not been defined.
`TypeError`	When an operation is applied to the wrong data type (e.g., adding a number and a string).
`ValueError`	When a function receives the right type but an invalid value (e.g., int("abc")).
`IndexError`	When you try to access a list index that does not exist.
`AttributeError`	When you try to access an attribute or method that does not exist on an object.
`KeyError`	When you try to access a dictionary key that does not exist.

Quick Tips for Beginners

- Always put risky code (user input, division, file access) inside a try block.
- Catch specific exceptions (ZeroDivisionError, TypeError) before using a bare except — it makes debugging easier.
- Use else to run code only when everything in try succeeded with no errors.
- Use finally for cleanup code that must always run (e.g. closing a file or database connection).
- Use raise to manually trigger an error when a value is logically wrong, even if Python wouldn't raise one automatically.
- The 'as e' in 'except ValueError as e' lets you access the error message and print it.

Practice Exercises

Try these exercises to test your understanding:

Exercise 1:

Write a function safe_divide(a, b) that uses try/except to handle ZeroDivisionError and TypeError. Test it with divide(10, 2), divide(10, 0), and divide(10, "x").

Exercise 2:

Ask the user to enter a number using input(). Use try/except to catch a ValueError if they type something that is not a number (like "hello"). Print a friendly message if they do.

Exercise 3:

Create a list of 5 items. Ask the user for an index. Use raise IndexError with a custom message if the index is out of range. Wrap it in try/except so the program does not crash.

Exercise 4 (Challenge):

Write a function check_password(password) that raises a ValueError if the password is shorter than 8 characters. Wrap the call in try/except/else/finally — print "Password accepted" in else and "Check complete" in finally.